Tarea 51

Nombre: Yamil Salomón Saldaña

Módulo: Análisis de Regresión

Análisis de Regresión Lineal Múltiple mediante Matrices

Objetivo: Construir un modelo predictivo para pronosticar el precio de una vivienda a partir del conjunto de datos kc_house_data.csv. El desarrollo incluye la selección de variables independientes, el cálculo matricial manual para obtener los estimadores de la Ecuación Normal, la evaluación de la bondad de ajuste y la posterior validación comparativa mediante un reporte automatizado.

1. Carga e Inspección de Datos

Se importan las librerías base para manejo de datos (pandas), operaciones matriciales (numpy) y validación estadística (statsmodels). Se carga el conjunto de datos para verificar su integridad y dimensiones iniciales

```python
import pandas as pd
import numpy as np
import statsmodels.api as sm

# Carga del conjunto de datos
df = pd.read_csv('kc_house_data.csv')

# Inspección inicial de la estructura
print("--- RESUMEN DEL CONJUNTO DE DATOS ---")
print(f"Total de observaciones: {df.shape[0]}")
print(f"Total de variables: {df.shape[1]}")
print(f"Valores nulos en el dataset: {df.isnull().sum().sum()}")
print("\nPrimeros registros:")
display(df.head(3))
```

```
--- RESUMEN DEL CONJUNTO DE DATOS ---
Total de observaciones: 21613
Total de variables: 21
Valores nulos en el dataset: 0

Primeros registros:
```

	id	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view	...	grade	sqft_above	sqft_basement	yr_built	yr_ren
0	7129300520	20141013T000000	221900.0	3	1.00	1180	5650	1.0	0	0	...	7	1180	0	1955	
1	6414100192	20141209T000000	538000.0	3	2.25	2570	7242	2.0	0	0	...	7	2170	400	1951	
2	5631500400	20150225T000000	180000.0	2	1.00	770	10000	1.0	0	0	...	6	770	0	1933	

3 rows × 21 columns

2. Selección de Variables y Construcción de la Matriz X e Y

Para construir el modelo general $Y = X\beta + e$, es necesario aislar nuestra variable dependiente y seleccionar un subconjunto lógico de variables independientes.

Se excluyen variables identificadoras o no lineales como id, date, zipcode, lat y long. Se seleccionan características intrínsecas de la propiedad: bedrooms, bathrooms, sqft_living, sqft_lot, floors, condition, grade y yr_built.

Matemáticamente, para que el modelo pueda calcular el intercepto (β_0, el valor base cuando todas las variables independientes son cero), es un requisito algebraico añadir una columna constante de unos (1) a la matriz X.

```python
# 1. Selección de variables independientes
cols_X = ['bedrooms', 'bathrooms', 'sqft_living', 'sqft_lot', 'floors', 'condition', 'grade', 'yr_built']

# 2. Construcción del Vector Y (Precio)
y = df['price'].values.reshape(-1, 1)

# 3. Construcción de la Matriz X
X_raw = df[cols_X].values

# 4. Adición de la columna de unos para el cálculo del Intercepto
unos = np.ones((X_raw.shape[0], 1))
X = np.append(unos, X_raw, axis=1)

print("--- DIMENSIONES DE LAS MATRICES ---")
print(f"Matriz X (con columna de unos): {X.shape}")
print(f"Vector Y: {y.shape}")
print("\nPrimer registro de la Matriz X (debe iniciar con 1.0):")
print(X[0])
```

```
--- DIMENSIONES DE LAS MATRICES ---
Matriz X (con columna de unos): (21613, 9)
Vector Y: (21613, 1)

Primer registro de la Matriz X (debe iniciar con 1.0):
[1.000e+00 3.000e+00 1.000e+00 1.180e+03 5.650e+03 1.000e+00 3.000e+00
 7.000e+00 1.955e+03]
```

3. Cálculo Matricial de Estimadores (Ecuación Normal)

Para encontrar el vector de coeficientes b que minimiza la suma de los residuales al cuadrado, se aplica el método de Mínimos Cuadrados Ordinarios (OLS) de forma manual mediante su notación matricial. La expresión algebraica a resolver es la Ecuación Normal:

$$b = (X^T X)^{-1} X^T Y$$

Este procedimiento requiere la transposición de la matriz X, su multiplicación por la matriz original, el cálculo de su matriz inversa mediante álgebra lineal y, finalmente, la multiplicación por la matriz transpuesta y el vector Y.

```python
# 1. Transpuesta de X
X_T = X.T

# 2. Multiplicación de X transpuesta por X
X_T_X = np.dot(X_T, X)

# 3. Cálculo de la matriz inversa
X_T_X_inv = np.linalg.inv(X_T_X)

# 4. Resolución del vector de coeficientes (b)
b = np.dot(np.dot(X_T_X_inv, X_T), y)

# Presentación de los resultados
nombres_coeficientes = ['Intercepto (Beta 0)'] + cols_X

print("--- VECTOR DE COEFICIENTES (b) CALCULADOS MANUALMENTE ---")
for nombre, coef in zip(nombres_coeficientes, b):
    # Formateamos a 4 decimales para mayor legibilidad
    print(f"{nombre}: {coef[0]:,.4f}")
```

```
--- VECTOR DE COEFICIENTES (b) CALCULADOS MANUALMENTE ---
Intercepto (Beta 0): 6,998,537.2939
bedrooms: -49,278.6922
bathrooms: 52,633.1348
sqft_living: 188.0648
sqft_lot: -0.2413
floors: 21,458.4360
condition: 19,183.0239
grade: 130,033.0441
yr_built: -4,000.1267
```

4. Evaluación de la Bondad de Ajuste: Suma de Cuadrados y R^2

Para determinar qué proporción de la varianza en el precio de las viviendas es explicada por nuestro modelo, calculamos la descomposición de la varianza mediante las sumas de cuadrados:

- **TSS (Suma Total de Cuadrados):** Mide la dispersión total de los precios reales respecto a su media.

$$TSS = \sum(Y_i - \bar{Y})^2$$

- **RSS (Suma de Cuadrados de los Residuales):** Representa la variabilidad que el modelo no logra explicar (el error).

$$RSS = \hat{e}^T\hat{e}$$

- **ESS (Suma Explicada de Cuadrados):** La variabilidad que sí explica el modelo.

$$ESS = TSS - RSS$$

Finalmente, calculamos el Coeficiente de Determinación (R^2) y su versión ajustada, la cual es crucial en modelos múltiples ya que penaliza la inclusión de variables que no aportan poder predictivo real:

$$\bar{R}^2 = 1 - (RSS / (n - (k + 1))) / (TSS / (n - 1))$$

```python
# 1. Calcular las predicciones del modelo (Y_estimada)
y_pred = np.dot(X, b)

# 2. Calcular los residuales (Errores = Real - Predicción)
residuales = y - y_pred

# 3. Cálculo manual de las Sumas de Cuadrados
RSS = np.sum(residuales**2)
TSS = np.sum((y - np.mean(y))**2)
ESS = TSS - RSS

# 4. Cálculo del R Cuadrado
R2 = 1 - (RSS / TSS)

# 5. Cálculo del R Cuadrado Ajustado
n = X.shape[0]      # Total de filas (21613)
k = X.shape[1] - 1  # Total de variables independientes sin el intercepto (8)
R2_adj = 1 - ((RSS / (n - k - 1)) / (TSS / (n - 1)))

# Presentación de Métricas
print("--- MÉTRICAS DE BONDAD DE AJUSTE (CÁLCULO MANUAL) ---")
print(f"Suma Total de Cuadrados (TSS): {TSS:,.2f}")
print(f"Suma de Cuadrados Residuales (RSS): {RSS:,.2f}")
print(f"Coeficiente de Determinación (R^2): {R2:.4f}")
print(f"R^2 Ajustado: {R2_adj:.4f}")
```

```
--- MÉTRICAS DE BONDAD DE AJUSTE (CÁLCULO MANUAL) ---
Suma Total de Cuadrados (TSS): 2,912,916,761,921,299.00
Suma de Cuadrados Residuales (RSS): 1,114,849,623,441,132.75
Coeficiente de Determinación (R^2): 0.6173
R^2 Ajustado: 0.6171
```

5. Validación Automatizada y Análisis de Relevancia (Statsmodels)

Para validar la precisión de nuestros cálculos matriciales y evaluar la significancia estadística individual de cada variable, contrastamos nuestros resultados con la librería statsmodels.

El objetivo es analizar los **P-values** (P > |t|). En el estándar de la industria, un P-value mayor a 0.05 indica que la variable no tiene significancia estadística y debe ser eliminada para optimizar el modelo y evitar ruido.

```python
# 1. Preparación de la matriz X con la constante usando statsmodels
X_df_sm = sm.add_constant(df[cols_X])

# 2. Ajuste del modelo mediante OLS (Mínimos Cuadrados Ordinarios)
modelo_automatizado = sm.OLS(y, X_df_sm).fit()

# 3. Impresión del reporte automatizado
print(modelo_automatizado.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.617
Model:                            OLS   Adj. R-squared:                  0.617
Method:                 Least Squares   F-statistic:                     4355.
Date:                Mon, 13 Apr 2026   Prob (F-statistic):               0.00
Time:                        18:13:59   Log-Likelihood:            -2.9723e+05
No. Observations:               21613   AIC:                         5.945e+05
Df Residuals:                   21604   BIC:                         5.945e+05
Df Model:                           8
Covariance Type:            nonrobust
==================================================================================
                     coef    std err          t      P>|t|      [0.025      0.975]
----------------------------------------------------------------------------------
const        6.999e+06   1.34e+05     52.046      0.000    6.73e+06    7.26e+06
bedrooms    -4.928e+04   2113.039    -23.321      0.000   -5.34e+04   -4.51e+04
bathrooms    5.263e+04   3572.979     14.731      0.000    4.56e+04    5.96e+04
sqft_living   188.0648      3.405     55.236      0.000     181.391     194.738
sqft_lot       -0.2413      0.038     -6.321      0.000      -0.316      -0.167
floors       2.146e+04   3587.469      5.981      0.000    1.44e+04    2.85e+04
condition    1.918e+04   2582.505      7.428      0.000    1.41e+04    2.42e+04
grade          1.3e+05   2230.917     58.287      0.000    1.26e+05    1.34e+05
yr_built    -4000.1267     69.131    -57.863      0.000   -4135.629   -3864.624
==============================================================================
Omnibus:                    17327.388   Durbin-Watson:                   1.984
Prob(Omnibus):                  0.000   Jarque-Bera (JB):          1208183.515
Skew:                           3.358   Prob(JB):                         0.00
Kurtosis:                      39.007   Cond. No.                     3.84e+06
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
[2] The condition number is large, 3.84e+06. This might indicate that there are
strong multicollinearity or other numerical problems.
```

6. Conclusiones y Ecuación Final del Modelo

1. Cálculos Manuales vs. Automáticos:

Comprobamos que hacer las multiplicaciones de matrices "a mano" en Python nos dio exactamente los mismos resultados que usar las herramientas automáticas de la librería statsmodels. Esto demuestra que el motor matemático que construimos paso a paso funciona a la perfección y es confiable.

2. ¿Qué tan exacto es el modelo y qué variables importan?

Nuestro "termómetro de confianza" (el R^2 Ajustado) marcó **0.614**. En términos de negocio, esto significa que las características que seleccionamos logran explicar el **61.4%** de los motivos por los cuales el precio de una casa sube o baja en la vida real.

Además, al revisar la tabla automática, vimos que el puntaje de relevancia (P-value) de *todas* nuestras variables fue de **0.000**. Como la regla dice que todo lo menor a 0.05 es útil, **no fue necesario eliminar ninguna variable**. Todas las características que elegimos (habitaciones, baños, metros, antigüedad, etc.) realmente importan para predecir el precio.